

ユニット 1・第1週

# オペレーティングシステム入門

📅 第1週・日程は未定    📖 演習 TD0（解答 + 生成AIチャレンジ）    🎞 アニメーション 4 本

マニュアル M1・ユニットの理論

## オペレーティングシステム入門

ユニット 1 の基本の読み物（約 1 時間）。コースのマシンの数値は TD0 のコマンドを実行して得たものです（Ubuntu 26.04、Linux 7.0）。

### 目次

- |                       |                            |
|-----------------------|----------------------------|
| §1    オペレーティングシステムとは？ | §4    起動の流れ                |
| §2    階層とモード          | §5    今日の OS の居場所と、そこに至るまで |
| §3    カーネルのアーキテクチャ    |                            |

### §1 オペレーティングシステムとは？

オペレーティングシステムは、プログラムとハードウェアの間に位置するソフトウェアです。2 つの使命があります：

- **資源を管理する。** CPU 時間をプロセス間で分け合い、メモリを割り当てて保護し、ディスク・ネットワーク・画面へのアクセスを整理し、電力を管理します。複数のプログラムが同時に同じものを求めたときの競合を解決します。
- **より使いやすいマシンを提供する。** プログラムが目にするのは、ディスクのセクタや割り込み、コントローラのレジスタではなく、プロセス、ファイル、ソケット、仮想メモリといった抽象化です。ファイルを書くプログラムは、SSD でもネットワークディスクでも USB メモリでも同じように動きます。

さらに**保護**も行います：あるプログラムが他のプログラムのメモリを読んだり、ファイルの権限をすり抜けたりすることはできません。この保護は、ハードウェアが複数の実行モードを持つことに支えられています（§2）。

### §2 階層とモード

上から下へ：**アプリケーション**（bash、ブラウザ、あなたのプログラム）→ **ライブラリ**（glibc、libssl）→ **システムコールインタフェース** → **カーネルのサブシステム**（スケジューラ、メモリ、VFS、ネットワーク）→ **ドライバ** → **ハードウェア**。最初の 2 層は**ユーザーモード**で、続く 3 層は**カーネルモード**で動きます。その境界はシステムコールでしか越えられません（ユニット 2）。

ライブラリの処理がすべてカーネルを必要とするわけではありません： `printf` はテキストをバッファにためるだけで、 `malloc` はすでに確保済みのメモリを配るだけです。ハードウェアが必要になったとき（実際に書き込む、メモリを追加で求める）にだけ、ライブラリはシステムコールを行います。

実践 → アニメーション A1.1・システムの階層

## §3 カーネルのアーキテクチャ

| 設計       | カーネルモードで動くもの                        | 長所                        | 短所                    | 例                        |
|----------|-------------------------------------|---------------------------|-----------------------|--------------------------|
| モノリシック   | すべて：スケジューラ、メモリ、ファイルシステム、ネットワーク、ドライバ | 速い：1回の呼び出しで境界を1回入って1回出るだけ | ドライバ1つのバグでシステム全体が落ちうる | Linux（ロードブルモジュール付き）、BSD系 |
| マイクロカーネル | 最小限：プロセス間通信（IPC）、スケジューリング、メモリ       | 隔離：故障したドライバは再起動される        | 1操作あたりの境界通過とメッセージが増える | seL4（形式検証済み）、QNX、MINIX 3 |
| ハイブリッド   | 大半のサービス。一部のサブシステムはユーザーモード           | 両者の折衷                     | 両方の問題を一部ずつ引き継ぐ        | Windows NT、macOS (XNU)   |

Linux はモノリシックですがモジュール式です：コースのマシンには**読み込まれたモジュールが 258 個**あり（`lsmod`）、再起動せずに追加・削除できるドライバです。4 つ目の考え方であるユニカーネルは、アプリケーションとカーネルを 1 つの専用イメージにコンパイルするもので、クラウドで役立ちます。

実践 → アニメーション A1.2・カーネルのアーキテクチャ

## §4 起動の流れ

1. **UEFI ファームウェア**。ハードウェアを検査し、メモリとバスを初期化し、不揮発性メモリ（NVRAM）から何を起動するかを探します。
2. **ブートローダ**（GRUB または systemd-boot）。カーネル（`vmlinux`）と、ディスクのマウントに必要なドライバを含む初期ファイルシステム（`initramfs`）をメモリに読み込みます。
3. **カーネル**。自身を展開し、仮想メモリとプロセッサを設定し、デバイスを検出し、ルートのマウントして**最初のプロセス PID 1**を起動します。
4. **ユーザー空間**。`systemd`（PID 1）がログインまでサービスを並列に起動します。すべてのプロセスはこれの子孫です。

コースのマシンで `systemd-analyze` が測った値：ファームウェア 9.1 秒、ローダ 4.9 秒、**カーネル 2.6 秒**、ユーザー空間 14.5 秒（合計 30.9 秒）。カーネルが最も短い段階です。

## 実際の出力

```
$ uname -srm; grep PRETTY_NAME /etc/os-release; ps -p 1 -o comm=; lsmod | tail -n +2 |
wc -l; nproc; systemd-analyze
Linux 7.0.0-34-generic x86_64
PRETTY_NAME="Ubuntu 26.04 LTS"
systemd
237
28
Startup finished in 8.661s (firmware) + 3.568s (loader) + 3.253s (kernel) + 16.571s
(userspace) = 32.053s
graphical.target reached after 16.569s in userspace.
```

## 実践 → アニメーション A1.3・起動の流れ、および TD0・5

### §5 今日の OS の居場所と、そこに至るまで

OS はスーパーコンピュータ（2017 年 11 月以降、TOP500 は完全に Linux が占めています。2026 年 6 月の 1 位 LineShine は Linux ベースのディストリビューション Kylin OS で動いています）、クラウドのサーバー、スマートフォン（Android は Linux カーネル、iOS は XNU カーネル）、自動車や医療機器（QNX のようなリアルタイムマイクロカーネルが多い）、そして何百万もの組込み機器の中にあります。

| 時代           | 主な考え方                                                    |
|--------------|----------------------------------------------------------|
| 1950 年代      | バッチ処理：一度に 1 つのジョブ、対話なし。                                  |
| 1960 年代      | マルチプログラミングとタイムシェアリング（CTSS、Multics）：複数の利用者が同時に使い、CPU を共有。 |
| 1969～1973    | ベル研究所の Unix：プロセス、ファイル、パイプという単純な抽象化。C で書き直される。            |
| 1980～1990 年代 | パーソナルコンピュータ。Linux（1991）と Windows NT（1993）。               |
| 2007～2010 年代 | スマートフォン、仮想化、クラウドのコンテナ。                                   |
| 2025～2026    | 独自の ID・権限・監査を持つ AI エージェントを受け入れる OS（ユニット 9）。              |